

Advanced Computer Networks

Selected Aspects of Packet Processing:

AQM, TC and XDP

Winter Term 2025/26

marius.feldmann@tu-dresden.de

Overview of sessions

- Six chapters for selected topics of advanced computer networks planned:
 - 1) 2025-10-16 - Selected Aspects of Packet Processing: AQM, TC and XDP
 - 2) 2025-10-23 - High-Availability for DC infrastructures
 - 3) 2025-10-30 - Network Virtualization and Software-defined Networking
 - 4) 2025-11-13 - Network Management
 - 5) 2025-12-04 – VPNs / Firewalling
 - 6) 2026-02-05 - Solar System Internet

Outline

- 1) Excurses with some basics about
 - Networking tools
 - Scapy
 - Network interfaces
 - Network namespaces
- 2) Introduction to Internet Traffic Engineering / Quality of Service
- 3) Active Queue Management
- 4) Traffic Control
 - tc tool
 - Classless and classful qdiscs
- 5) Overview of eBPF
- 6) eXpress Data Path
 - Motivation
 - Overview
 - Simple XDP example program
 - AF_XDP

Intention is to provide selected insights into L3 processing on hosts and routers and to present options to influence the processing.

Excursus: Useful Tools / Packets (Linux / *nix)

- **ip** (iproute2) - Show / manipulate routing, network devices, interfaces and tunnels
- **tcpdump** - Dump traffic on a network
- **nc** / netcat - Arbitrary TCP and UDP connections and listens
- **mtr** - A network diagnostic tool
- **ss** - Another utility to investigate sockets
- **iperf** - Perform network throughput tests
- **nmap** - Network exploration tool and security / port scanner
- **iptables** / **nft** - Administration tool for IPv4/IPv6 packet filtering and NAT
- Administration tool for packet filtering and classification

Excursus: Scapy

- Tool developed in Python for creating and manipulating packets and for packet sniffing
- Available as library to be embedded in own software (bindings for further languages exist) and offers interactive mode
- Packets created in the interactive mode can be send (send for I3+, sendp for I2) via Scapy and replies can be received and evaluated via sr

```

      aSPY//YASa
      apyyyyCY/////////YCa
      sY////////YSpcs   scpCY//Pp
ayp  ayyyyyySCP//Pp      syY//C
AYAsAYYYYYYYY//Ps      cY//S
      pCCCCY//p          cSSps y//Y
      SPPPP//a           pP///AC//Y
      A//A               cyP///C
      p///Ac             sC///a
      P///YCpc           A//A
      scccccp///pSP///p   p//Y
      sY////////y  caa      S//P
      cayCyayP//Ya        pY/Ya
      sY/PsY///YCc        aC//Yp
      sc  sccaCY//PCypaapyCP//YSs
           spCPY////////YPSps
           ccaacs

| Welcome to Scapy
| Version 2.4.3
|
| https://github.com/secdev/scapy
|
| Have fun!
|
| To craft a packet, you have to be a
| packet, and learn how to swim in
| the wires and in the waves.
|       -- Jean-Claude Van Damme
|

>>> pkt=IP(ttl=20,src="8.8.8.8")/TCP(dport=443)
>>> pkt[TCP].sport=4242
>>> pkt
<IP frag=0 ttl=20 proto=tcp src=8.8.8.8 |<TCP sport=4242 dport=https |>>
>>> send(pkt)
.
Sent 1 packets.
>>> ls(TCP)
sport      : ShortEnumField          = (20)
dport      : ShortEnumField          = (80)
seq        : IntField                = (0)
ack        : IntField                = (0)
dataofs    : BitField (4 bits)       = (None)
reserved   : BitField (3 bits)       = (0)
flags      : FlagsField (9 bits)     = (<Flag 2 (S)>)
window     : ShortField              = (8192)
chksum     : XShortField             = (None)
urgptr     : ShortField              = (0)
options    : TCPOptionsField         = (b'')
>>> 
```

Excursus: Network Interfaces (Linux)

- IP configuration is assigned to network interfaces (not to "hosts")
- Several types of network interfaces exist, see e.g. Manpage of ip-link (bond, gre, macsec, veth, vxlan, ...)
- For configuring a network interface, its name is typically used
- Simple approach used for many years:
 - Abbreviation for interface type plus index (ethX / wlanY, e.g. eth1, wlan0)
- Approach taken typically today: Consistent Network Device Naming
 - Avoids renaming if e.g. devices are replaced
 - Example: enp0s25 → ethernet interface (en), which is available via the PCI bus (p) at slot 25
 - For implementation, see https://github.com/systemd/systemd/blob/master/src/udev/udev-builtin-net_id.c

Excursus: Network Namespaces

- A Linux kernel namespace renders it possible to attach processes to specific kernel resources and to block access to resources not associated to this namespace
- A network namespace is a logical instance of a network stack including own network interfaces, routing tables or iptables rules
- Initially all processes share the same network namespace
- Processes can be started within network namespaces
- Network namespaces can be connected to each other using e.g. virtual ethernet interface pairs (veth interfaces)

```
$ ip netns add netOne           # Create network namespace „netOne“
$ ip netns add netTwo          # Create network namespace „netTwo“
$ ip link add veth1 netns netOne type \
    veth peer name veth2 netns netTwo # Create veth pair
$ ip netns exec netOne /bin/bash # Start process executing /bin/bash in
                                # network namespace "netOne"

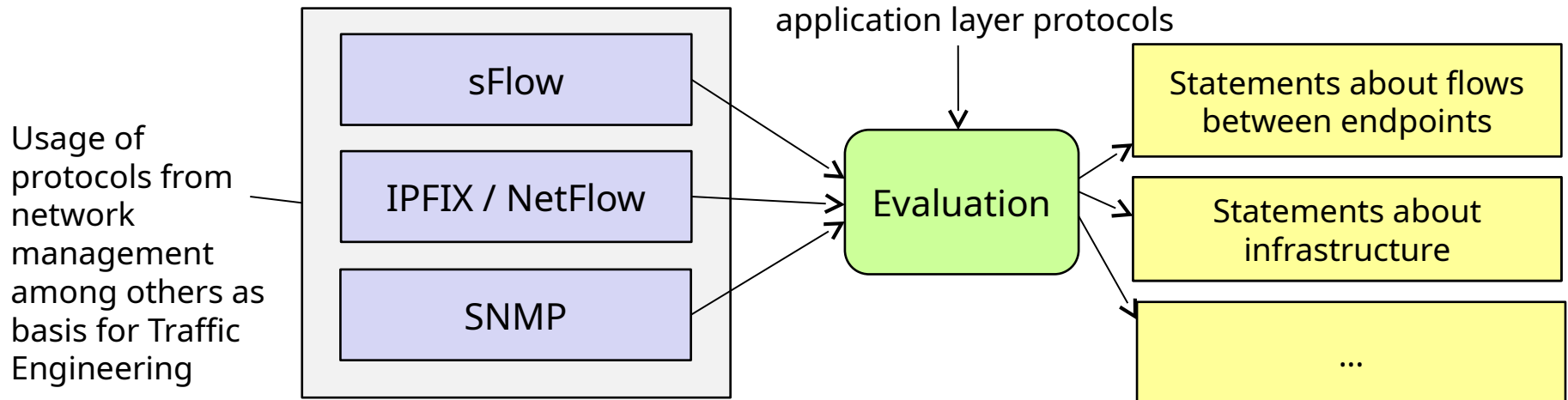
$ ip address show
1: lo: <LOOPBACK> mtu 65536 ...
2: veth1@if2: <BROADCAST,MULTICAST> mtu 1500...
   link/ether 96:97:d1:7c:42:f6 brd ff:ff:ff:ff:ff:ff link-netns netTwo
```

Traffic Engineering

- From RFC 9522: "Internet traffic engineering is defined as that aspect of Internet network engineering dealing with the issues of **performance evaluation and performance optimization** of operational IP networks. Traffic engineering encompasses the application of technology and scientific principles to the **measurement, characterization, modeling, and control of Internet traffic** [...]."

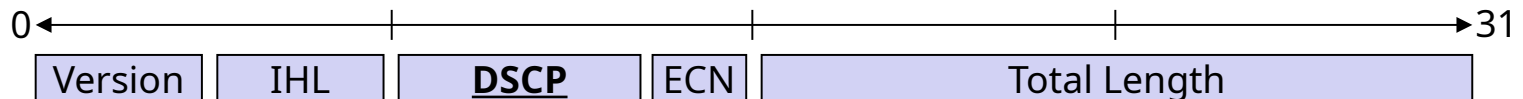
"A central goal of the service provider, therefore, is to enhance the emergent properties of the network while taking economic considerations into account. This is accomplished by addressing traffic-oriented performance requirements while utilizing network resources without excessive waste and in a reliable way. Traffic-oriented performance measures include **delay, delay variation, packet loss, and throughput.**"

- Estimation of e.g.:
 - Scalability of network
 - Implications of usage of further application layer protocols



Quality of Service (QoS)

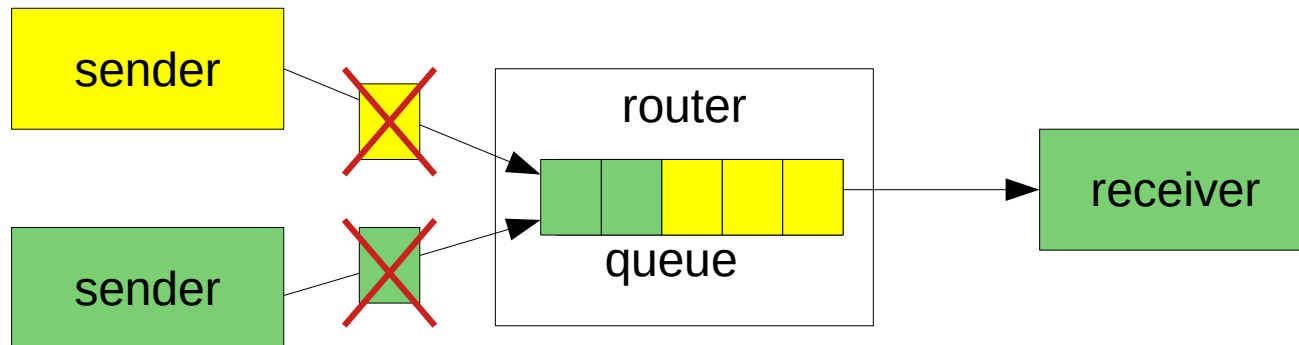
- From RFC 3198: „At a high level of abstraction, "Quality of Service" refers to the ability to deliver network services **according to the parameters specified in a Service Level Agreement**. "Quality" is characterized by service **availability, delay, jitter, throughput and packet loss ratio**. At a network resource level, "Quality of Service" refers to a set of capabilities that allow a service provider to prioritize traffic, control bandwidth, and network latency.“
- For IP networks:
 - Integrated Services (**IntServ**); see RFCs 2210, 2211, 2212
 - Reservation of resources per packet flow before packets are received
 - Dedicated protocol for reservation: Resource Reservation Protocol (RSVP)
 - Differentiated Services (**DiffServ**); see RFC 2475 and RFC 3260
 - Packets are mapped to classes by marking and forwarding is done based on these marks
 - Uses Differentiated Services Codepoints (DSCP) for IPv4 / IPv6
 - First 32 Bits from IPv4 header (DSCP field as described in RFC 2474):



- Based on the DSCP per-hop behavior (PHB) can be defined (e.g. expedited forwarding (RFC 3246) or assured forwarding (RFC 2597))

Active Queue Management (AQM) - Motivation

- Router manage internal packet queues
 - Queue management determines which packets are dropped or marked
 - Scheduling algorithm determines which packet is forwarded next



- "Tail drop" used for queues with a maximum amount of packet slots causes several issues (see RFC 7567):
 - Increase of RTTs
 - Synchronization of control loops
 - Packets of single flows are dropped
 - Packet bursts result in huge amounts of dropped packets
- Issues are even worse in the case of queues with many packet slots (see "bufferbloat" issue)

Bufferbloat

- "The culprit is bufferbloat, which is the existence of excessively large and frequently full buffers inside the network. ... Long delays from bufferbloat are frequently attributed incorrectly to network congestion, and this misinterpretation of the problem leads to the wrong solutions being proposed." (Gettys, J. and K. Nichols, "Bufferbloat: Dark Buffers in the Internet", Com. of the ACM, Volume 55, Issue 1, January 2012.)

Running ping while doing a speed test:

```
$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) Bytes an Daten.
64 Bytes von 8.8.8.8: icmp_seq=1 ttl=118 Zeit=24.4 ms
64 Bytes von 8.8.8.8: icmp_seq=2 ttl=118 Zeit=24.3 ms
64 Bytes von 8.8.8.8: icmp_seq=3 ttl=118 Zeit=22.6 ms
...
64 Bytes von 8.8.8.8: icmp_seq=22 ttl=118 Zeit=46.6 ms
64 Bytes von 8.8.8.8: icmp_seq=23 ttl=118 Zeit=23.7 ms
64 Bytes von 8.8.8.8: icmp_seq=24 ttl=118 Zeit=44.4 ms
...
64 Bytes von 8.8.8.8: icmp_seq=31 ttl=118 Zeit=103 ms
64 Bytes von 8.8.8.8: icmp_seq=32 ttl=118 Zeit=115 ms
...
64 Bytes von 8.8.8.8: icmp_seq=53 ttl=118 Zeit=23.9 ms
64 Bytes von 8.8.8.8: icmp_seq=54 ttl=118 Zeit=23.5 ms
^C
--- 8.8.8.8 Ping-Statistiken ---
54 Pakete übertragen, 54 empfangen, 0% packet loss, time 53073ms
rtt min/avg/max/mdev = 20.914/45.049/117.247/28.954 ms
```

Indicators for bufferbloat?

Issue could be caused by the first hop router – and/or by any router on the path.

Active Queue Management (AQM)

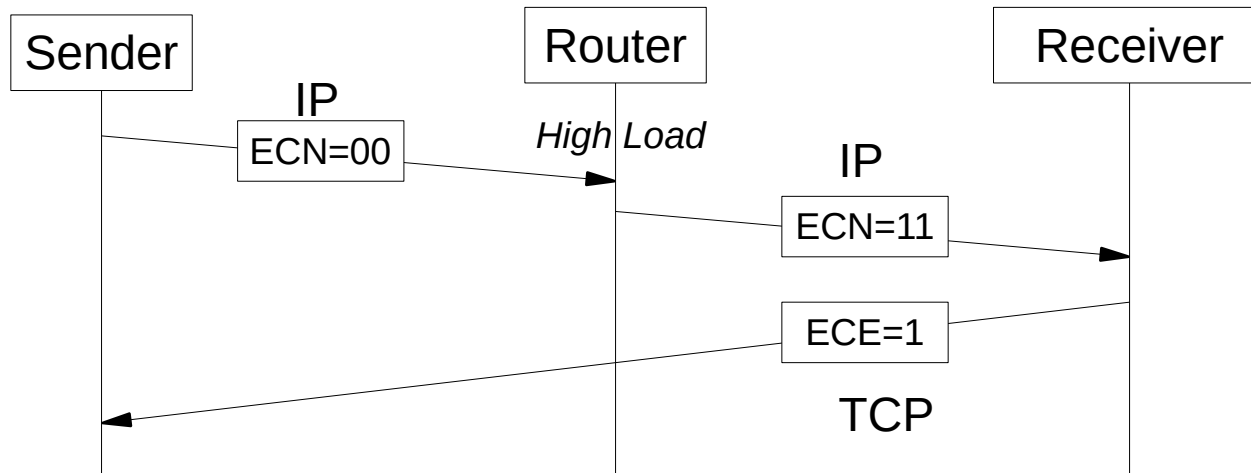
- Network scheduler uses AQM to ensure that the queue length is kept at a low level, thus reducing overload situations as well as latency (algorithms such as Random Early Detection (RED) or Controlled Delay (CoDel))
- Recommendations from RFC 7567 (see section 4):
 - *Network devices SHOULD implement some AQM mechanism to manage queue lengths, reduce end-to-end latency, and avoid lock-out phenomena within the Internet.*
 - *Deployed AQM algorithms SHOULD support Explicit Congestion Notification (ECN) as well as loss to signal congestion to endpoints.*
 - *AQM algorithms SHOULD NOT require tuning of initial or configuration parameters in common use cases.*
 - *AQM algorithms SHOULD respond to measured congestion, not application profiles.*
 - *AQM algorithms SHOULD NOT interpret specific transport protocol behaviors.*
 - *Congestion control algorithms for transport protocols SHOULD maximize their use of available capacity (when there is data to send) without incurring undue loss or undue round-trip delay.*
 - *Research, engineering, and measurement efforts are needed regarding the design of mechanisms to deal with flows that are unresponsive to congestion notification or are responsive, but are more aggressive than present TCP.*

Selected AQM Schemes

- **Random Early Detection (RED)** – RFC 2309
 - Two steps:
 - 1) Estimation of average queue size (either packets or number of bytes) is done using a simple exponentially weighted moving average
 - 2) Packets are dropped if the average queue size is between a minimum and maximum threshold with a probability that scales linearly from 0 to a defined maximum probability
 - Several configuration parameters have to be provided which are difficult to determine
 - IETF obsoleted recommendation to use RED as default AQM scheme
- **Controlled Delay (CoDel)** – RFC 8289
 - Queuing delay is calculated per packet (sojourn delay) within a specific time interval and it is checked if the minimum queuing delay during the interval is higher than a target delay (set to 5 ms for "normal" Internet traffic) resulting in dropping (or marking) the next packet (in this case, the interval is reduced in inverse proportion to the square root of the number of consecutive drops)
 - Only interval value as configuration parameter is required (default value is 100 ms)
- **Proportional Integral Controller Enhanced (PIE)** – RFC 8033
 - Calculates queuing latency (delay) and drops (or marks) packets based on a probability that depends on the delta between the calculated latency and a target latency (default is 15 ms) and on the fact if there is a trend of increasing or decreasing latency
 - Only two configuration parameters with provided defaults (AQM Latency Target and AQM Max Burst Allowance)

Explicit Congestion Notification (ECN)

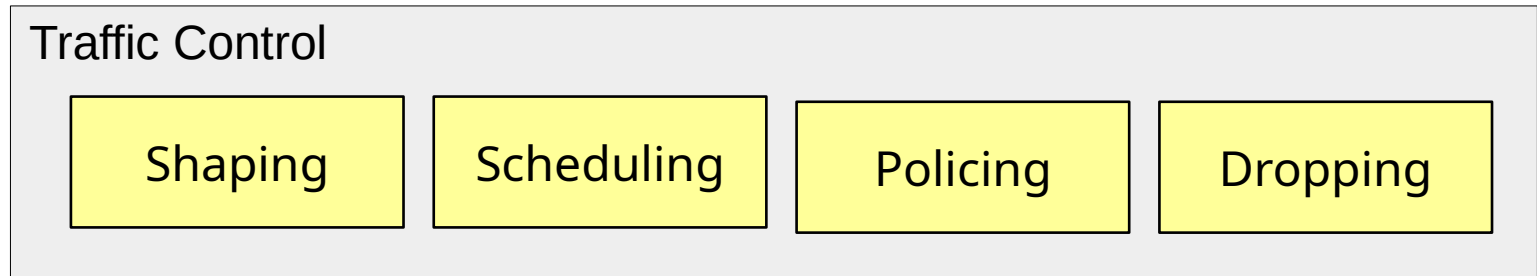
- Intention of ECN (described in RFC 3168) is to proactively address emerging congestion situations without dropping packets
- A router can mark IP packets to indicate emerging congestion situations
- Marking results in communication from receiver to sender on layer 4 (TCP or QUIC)



- ECN Echo is transmitted to sender, to signal high load in the network
- After adapting the congestion window, the receiver is informed (e.g. via CWR flag in TCP)

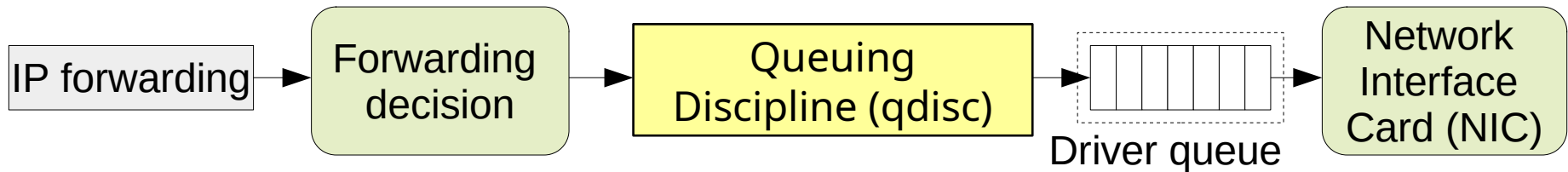
Traffic Control

- Describes methods and their implementation for managing queues for sending and receiving packets over a network interface
- According to the Manpage of `tc` command, traffic control covers four topics:



- Shaping: Controlling egress traffic based on a policy via e.g. reducing the available bandwidth - from RFC 2475: "Shapers delay some or all of the packets in a traffic stream in order to bring the stream into compliance with a traffic profile"
- Scheduling: Selection of packet(s) for forwarding from a set/list of packets
- Policing: Controlling the ingress traffic similar to shaping
- Dropping: Approach of selecting and deleting packets e.g. if the ingress or egress bandwidth is exceeded

Classless/Classful Queuing



- Each network interface has a qdisc for egress communication (egress/root qdisc) to which the kernel adds packets which are intended to be forwarded via this interface
- Packets are fetched from the qdisc and added to the interface's driver queue → decision which packet is handed over to the driver queue is done via qdisc
- Two types of qdiscs:
 - **Classless qdiscs:**
 - Are capable to drop packets, change their order or delay forwarding based on predefined algorithms
 - Do not support hierarchies of qdiscs
 - No classification of packets, no filters
 - **Classful qdiscs:**
 - Are capable to classify packets based on filters and type of service information
 - Support hierarchies of qdiscs

Classless Queuing

- Linux supports more than a dozen qdiscs
- Configuration of qdiscs is done via the tool tc (similar tool for BSD systems: dummynet)
- Configuration of a classless qdisc:

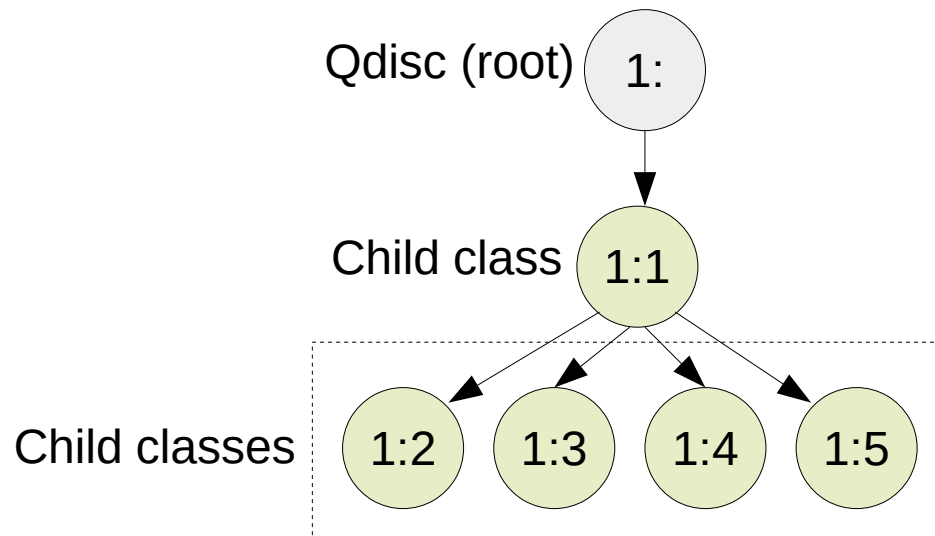
```
# tc qdisc add dev $DEVICE root $QDISC $PARAMETERS
```

- Examples for qdiscs:
 - Pfifo/bfifo: First-In-First-Out-Scheduling with the option to provide a maximum queue length (pfifo → packets; bfifo → bytes)
 - Sfq - Stochastic fairness queuing:
 - Based on a hash value derived from a packet (by default it is calculated based on the source/destination IP address and port number) a packet is added to a bucket
 - Packets are forwarded based on round robin → fair treatment of all flows
 - Tbf - Token bucket filter:
 - Offers option to specify a burst size and data rate plus latency which is applied on egress packets, e.g.:

```
# tc qdisc add dev enp0s25 root tbf rate 512kbit latency 10ms burst 2048
```

Classful Queuing

- If packet is added to classful qdisc, it will be associated to one of the qdisc's classes (usually based on filters)
- Each class has a default qdisc in case the qdisc is not configured (pfifo_fast)



- Qdiscs, classes and filters have an ID with a major and minor number separated by a colon

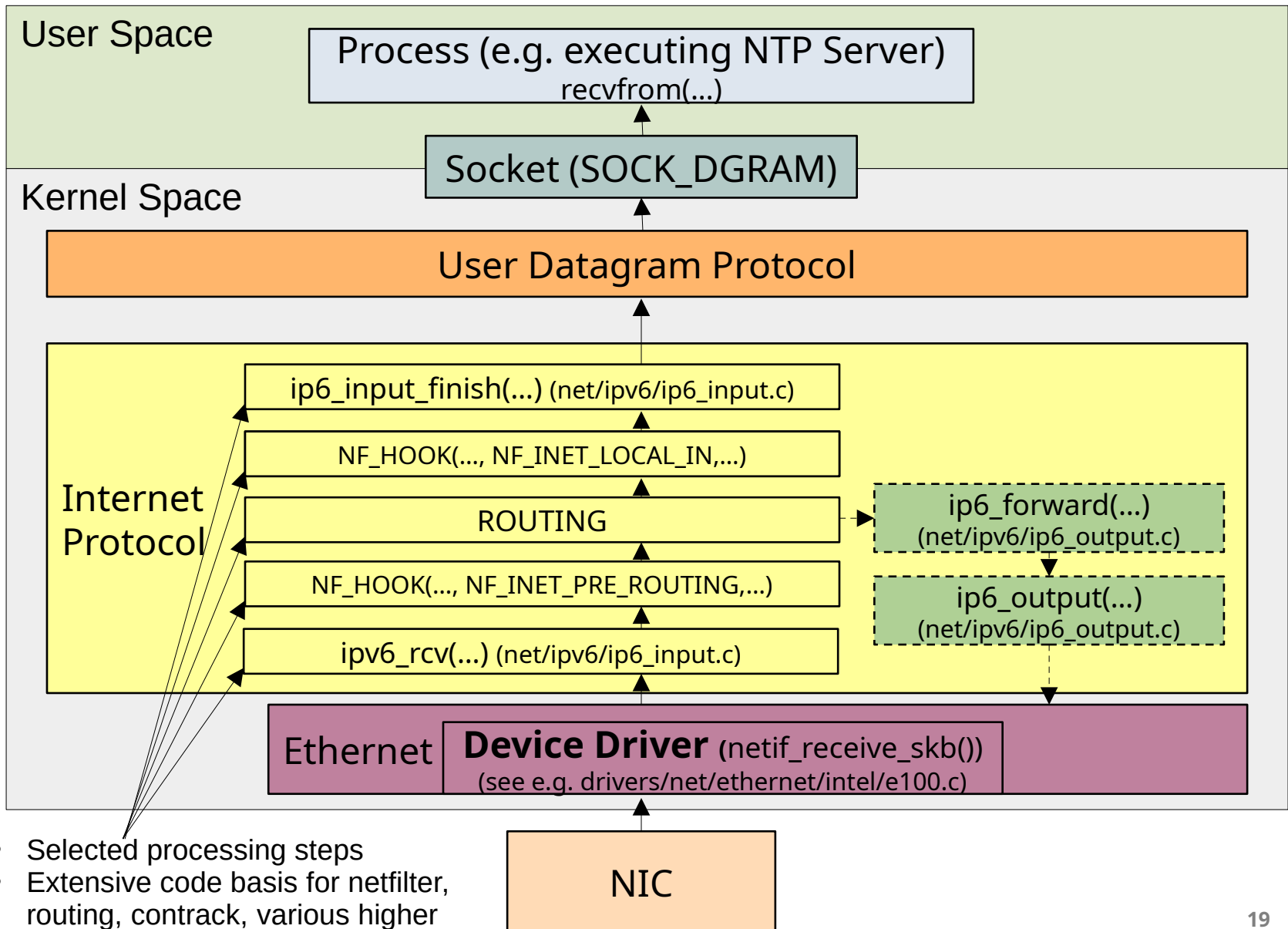
- Example for creating a class (htb - Hierarchical Token Bucket – is assumed):

```
# tc class add dev enp0s25 parent 1:1 classid 1:3 htb rate 100mbit \
    ceil 150mbit
```

- Example for a filter added to a class

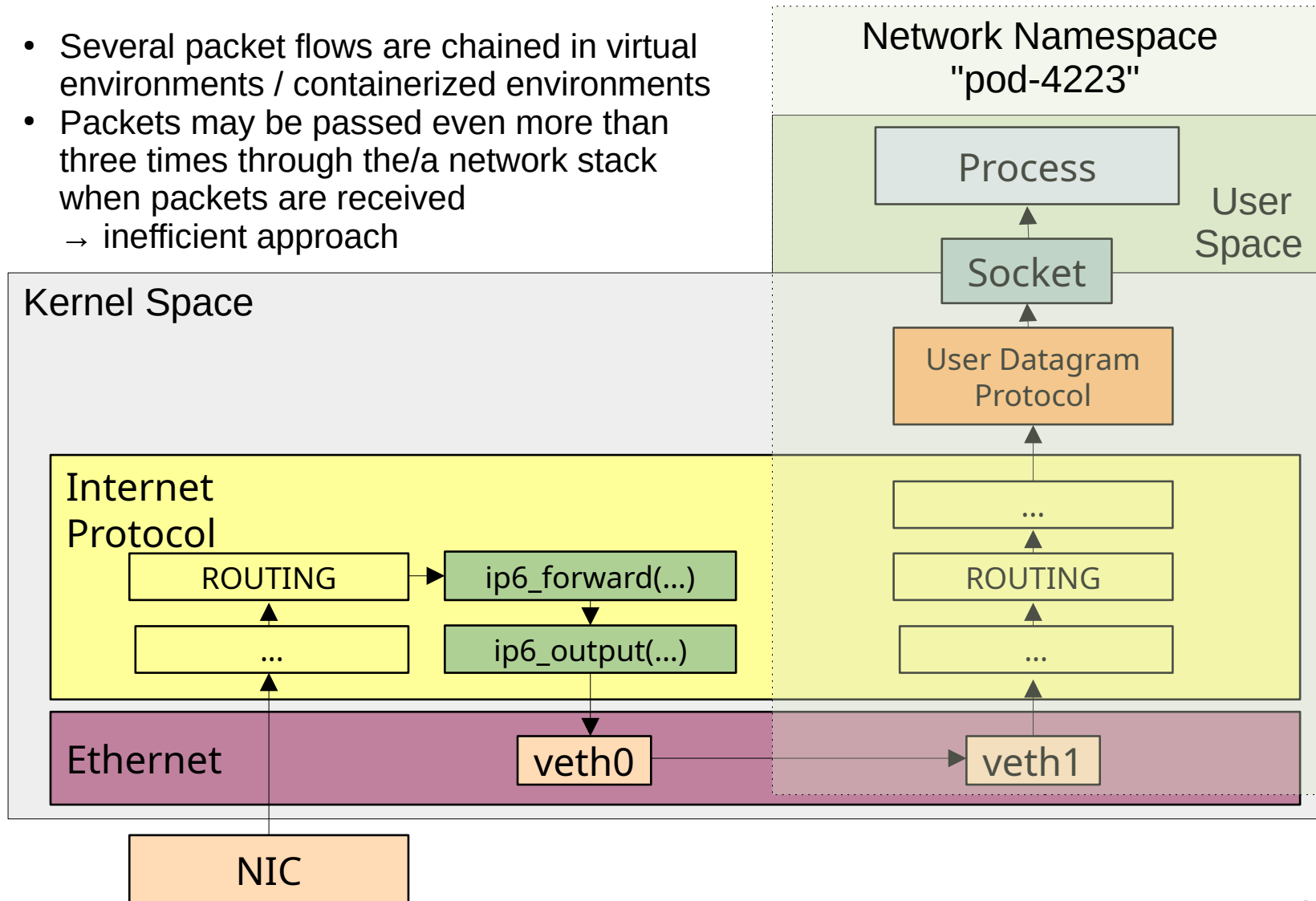
```
# tc filter add dev enp0s25 protocol ip parent 1:0 prio 0 u32 match \
    ip src 141.76.40.21 flowid 1:3
```

XDP Motivation – Simplified Packet-In-Flow



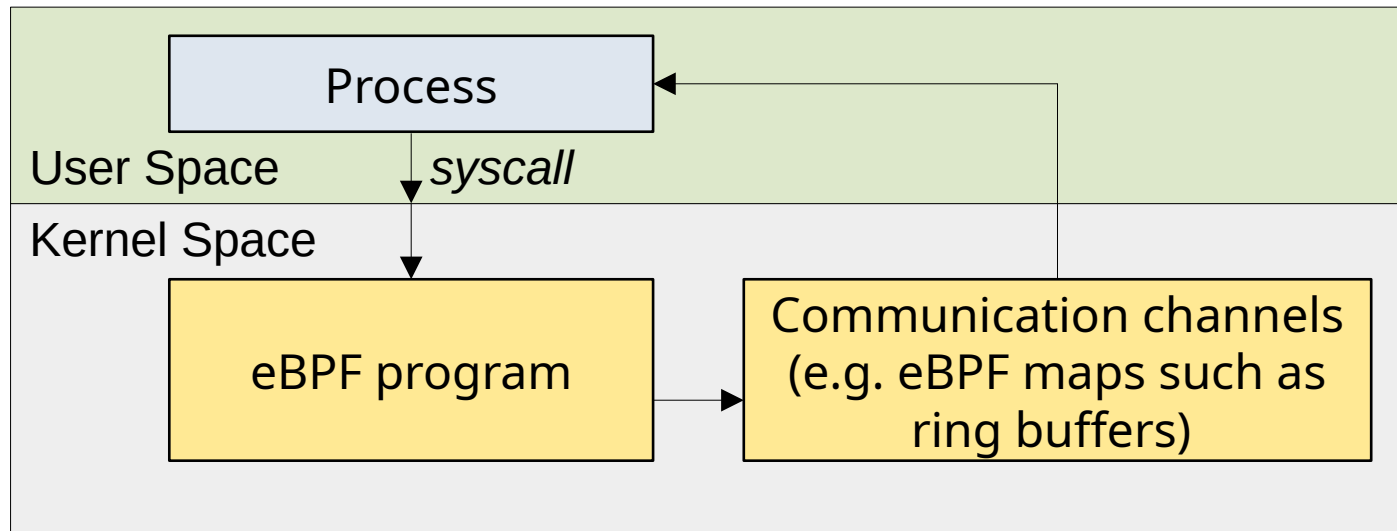
XDP Motivation – Packet Flow for a Containerized Application

- Several packet flows are chained in virtual environments / containerized environments
- Packets may be passed even more than three times through the/a network stack when packets are received
→ inefficient approach



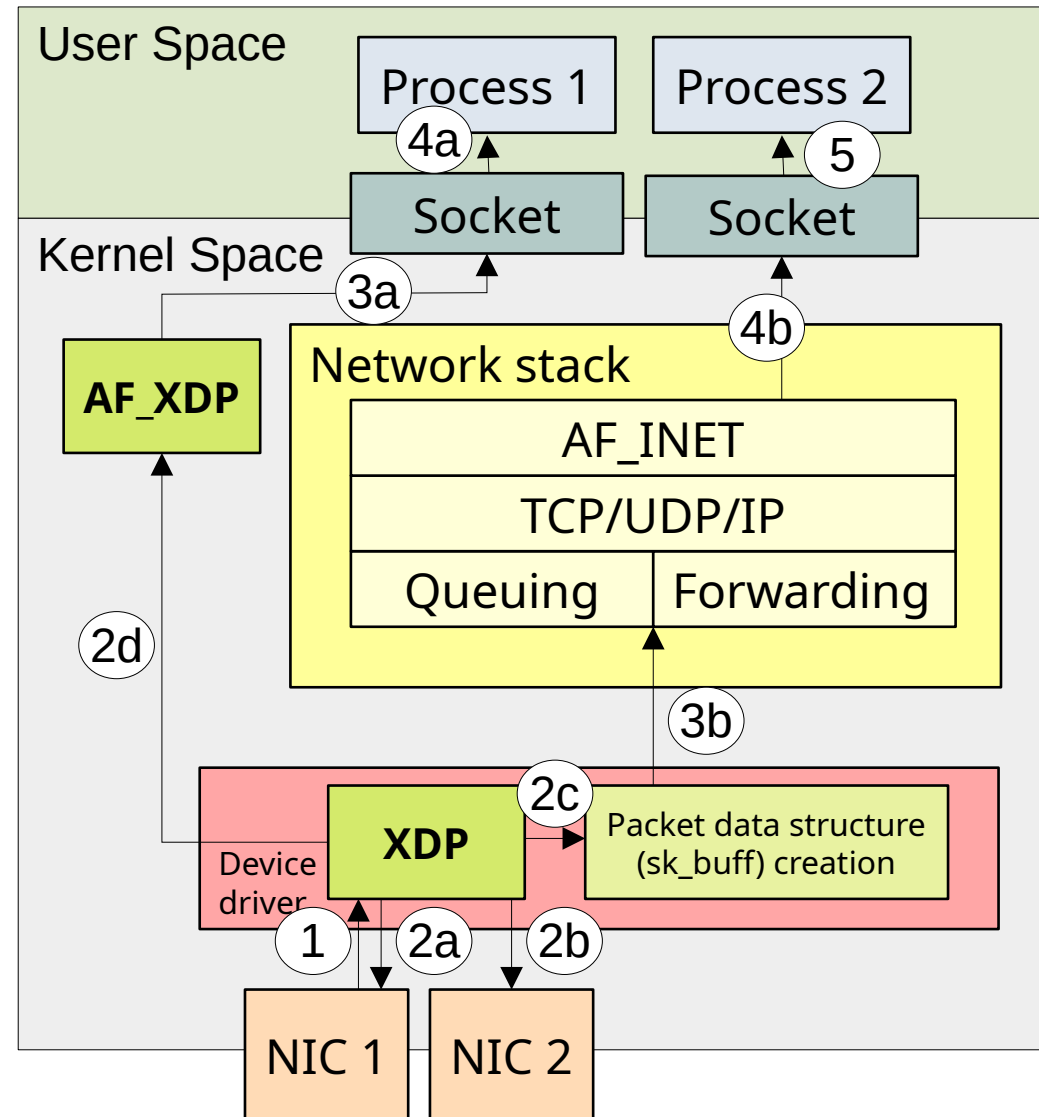
Overview of eBPF

- Providing extensions and patches to an operating system such as Linux is a lengthy process, e.g. it may take more than a year to get code changes "upstream" (afterwards, distributions have to integrate new versions)
- Kernel modules are one option to speed up this process, however they may cause instability of the system in case of bugs / errors in the module's source code
- eBPF solves both issues by providing an ad-hoc extension mechanism for the Linux kernel without any stability issues due to the eBPF verifier
- eBPF code is executed based on Just-in-Time compilation on a virtual machine within the kernel in case specific events are triggered (such as execution of a syscall or receiving a packet)



XDP overview

- The core idea of the eXpress Data Path (=XDP) is build upon a hook for executing eBPF programs located in the network interface device driver after packet reception (before sk_buff is created)
- If the device driver does not support XDP, a generic XDP implementation is provided within the network stack (loss of performance advantages, e.g. for testing purposes)
- XDP programs may be offloaded to a NIC
- Used for many high-performance network functions such as:
 - Load balancing
 - Firewalling
 - Intrusion Detection
 - Forwarding packets to containers



- Option 2e is to drop the packet
- Additional hook to BPF functionality is e.g. TC BPF within the network stack (not displayed)

Simple XDP example

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>

SEC("xdp")
int xdp_simple(struct xdp_md *ctx)
{
    return XDP_PASS;
}
char_license[] SEC("license") = "GPL";
```

- ELF (Executable and Linking Format) section named "xdp" is created and used as a reference to trigger the XDP program (see "sec xdp" in the ip command below)

1

- Struct contains package data (and meta data)

- Return value determines what should be done with the packet

2

```
$ clang -O2 -g -Wall -target bpf -c xdp_simple.c -o xdp_simple.o
```

3

```
# ip link set $INTERFACE xdpgeneric obj ./xdp_simple.o sec xdp
```

4

```
# ip a s $INTERFACE
4: $INTERFACE: <BROADCAST,MULTICAST,UP,LOWER_UP> \
mtu 1500 xdpgeneric/id:498 \
qdisc noqueue state UP group default qlen 1000
...
```

XDP selected data structures

- Example showing access to ttl header field



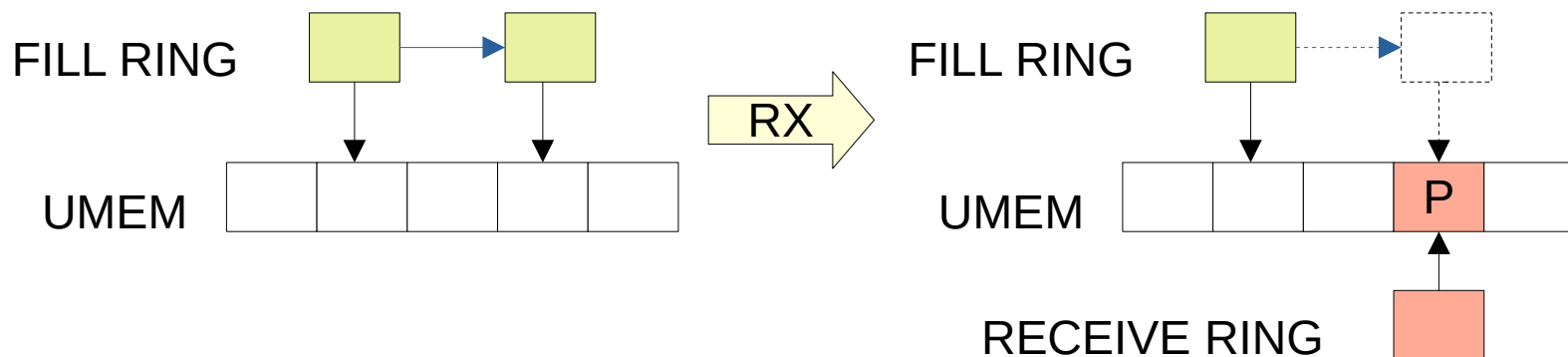
```
enum xdp_action {
    XDP_ABORTED = 0,
    XDP_DROP,
    XDP_PASS,
    XDP_TX,
    XDP_REDIRECT,
};
...
struct xdp_md {
    __u32 data;
    __u32 data_end;
    __u32 data_meta;
    ...
    __u32 ingress_ifindex;
    __u32 rx_queue_index;
    __u32 egress_ifindex;
};
```

```
...
void *end = (void *) (long) ctx->data_end;
void *frame = (void *) (long) ctx->data;
struct ethhdr *eth = frame;
if (frame + sizeof(struct ethhdr) > end) {
    return XDP_DROP;
}
void *pkt = frame + sizeof(struct ethhdr);
__u16 l3_proto = bpf_ntohs(eth->h_proto);
if (l3_proto == ETH_P_IP) {
    if (pkt + sizeof(struct iphdr) > end) {
        return XDP_DROP;
    }
    struct iphdr *ip = pkt;
    if (ip->ttl <= 1) {
        return XDP_PASS;
    }
    ...
}
```

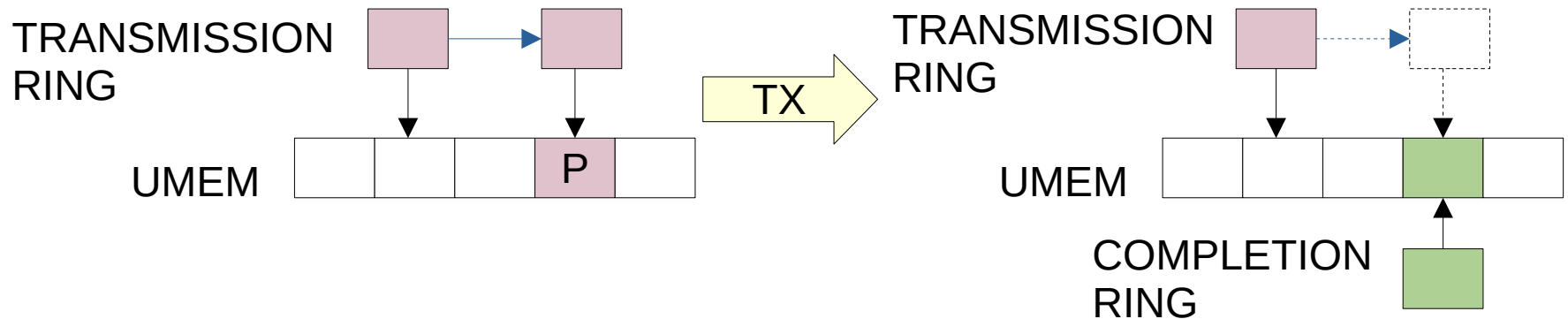
- Five verdicts determine what will be done with the treated packet
 - **XDP_ABORTED** → indicates an XDP program error plus packet is dropped; **XDP_DROP** → drop packet; **XDP_PASS** → send packet to network stack; **XDP_TX** → return packet on interface it has been received from; **XDP_REDIRECT** → send packet to different interface
- Member "data" of struct xdp_md points to the start of the L2 frame data; member "data_meta" can be used to store additional data about the packet (e.g. for communication between several XDP programs)

AF_XDP

- AF_XDP is an address family for a socket allowing user space programs to transmit and receive raw packets
- Used e.g. for implementing custom protocols or for software reducing DDoS impact
- Communication between kernel and user space is build upon single-producer/single-consumer (SPSC) descriptor ring queues
- UMEM memory area is preallocated for transferring packets
- Via a so called *fill ring* the user space indicates to which UMEM frame the kernel should add a packet
- As soon as a packet is added to a UMEM frame, the frame's descriptor is added to the so called *receive ring* via which the user space process can "gain back control" of the UMEM frame



AF_XDP



- *Completion ring* is used to hand over ownership of UMEM frames from kernel space to user space after transmission has been done
- Possible process to set up an AF_XDP socket:
 - 1) Create AF_XDP socket
 - 2) Create UMEM by providing frame (packet) size and frame amount
 - 3) Link UMEM to socket via *setsockopt*
 - 4) Allocate buffer space for four ring buffers
 - *XDP_TX_RING*
 - *XDP_RX_RING*
 - *XDP_UMEM_FILL_RING*
 - *XDP_UMEM_COMPLETION_RING*
 - 5) Map ring buffers into process memory
 - 6) Associate socket to network interface / hardware queue via *bind*

References

RFC about Traffic Engineering:
<https://tools.ietf.org/html/rfc9522>

RFC about AQM:
<https://tools.ietf.org/html/rfc7567>

RFC about ECN:
<https://tools.ietf.org/html/rfc3168>

Manpage of tc:
<https://linux.die.net/man/8/tc>

XDP tutorial:
<https://github.com/xdp-project/xdp-tutorial>

Introduction to AF_XDP :
https://docs.kernel.org/networking/af_xdp.html